

24/07/2026

http://localhost:1257/webapp/

USERFUNCTION - Função de usuário

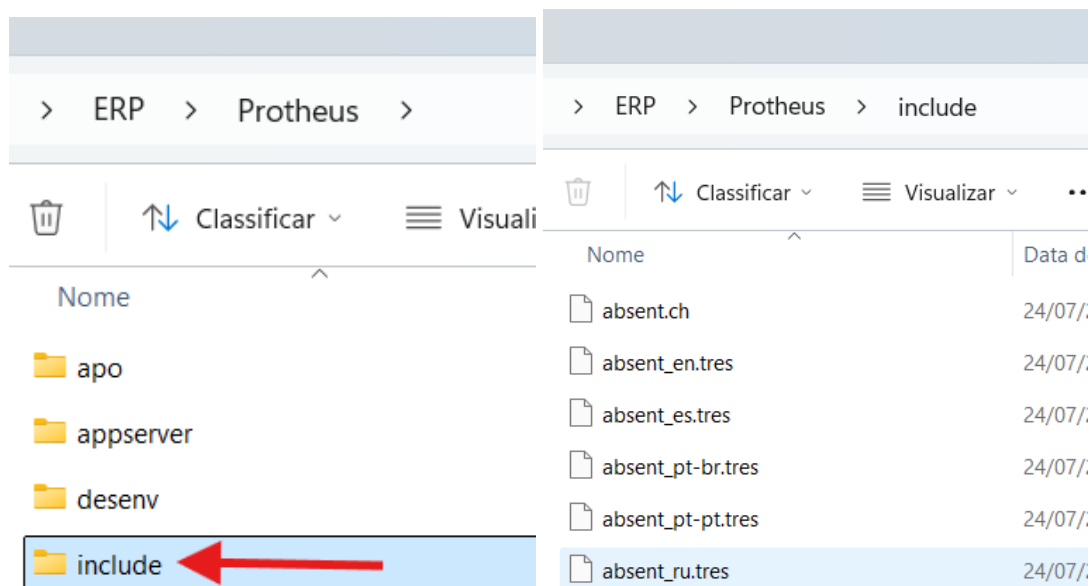
Para compilar

CTRL + F9

SIGAADV

Recursos Necessários

Includes na pasta do Protheus



Incluir um novo servidor

Server Name: test

Address: localhost

Port: 1257 (olhar o appserver.ini Protheus)

Include directory: caminho da pasta onde estão os includes

New Server

Protheus (Adv/PL) ▾

test

localhost

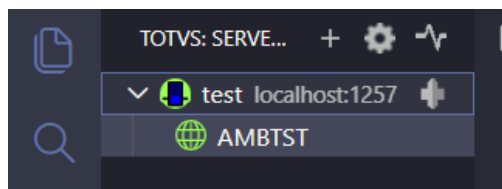
1257

Includes directory***Allow multiple directories**

C:\TOTVS\ERP\Protheus\include

Choose File No file chosen

SAVE SAVE/CLOSE



Ao conectar, Preencher o usuário e senha, o usuário é o admin.

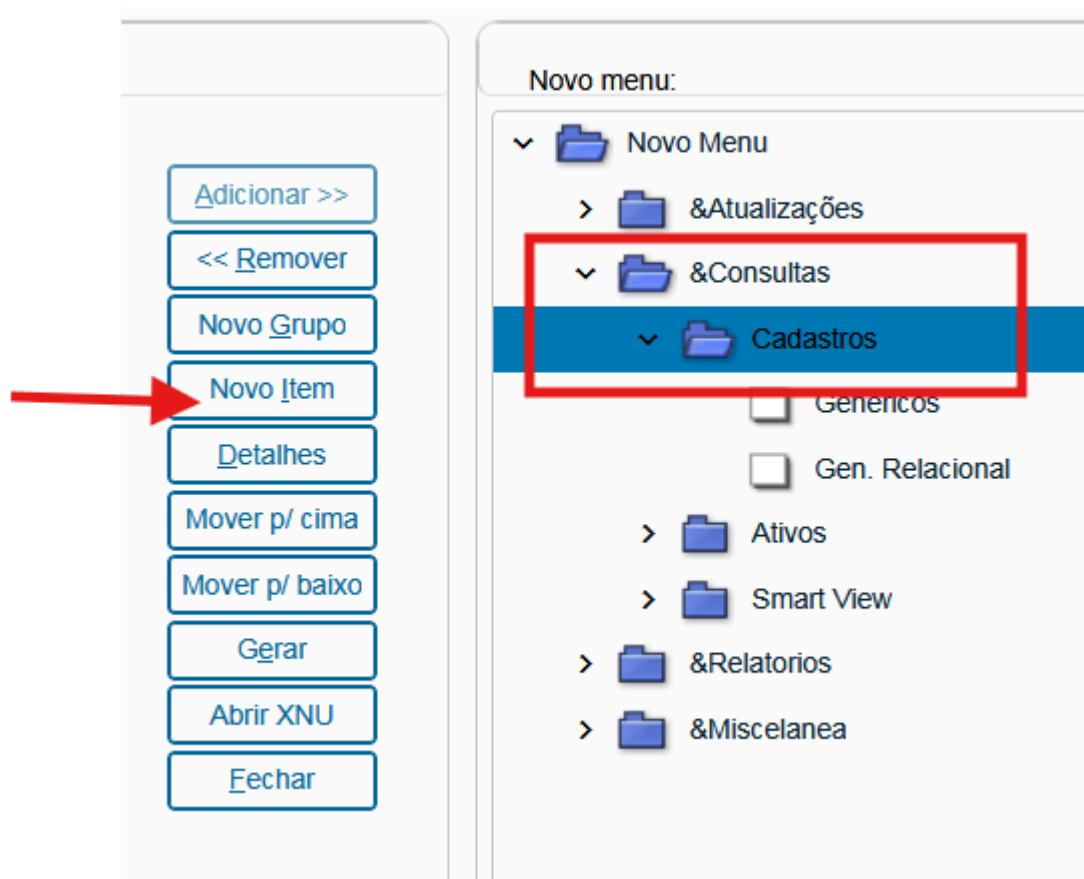
Como Testar

Criar Menu

Selecionar o Módulo a ser substituído

Menu	
☐	Aprovacao
☐	Arquitetura Organizacional
☒	Ativo Fixo
☐	Auto Peças
☐	Automação Coleta de Dados
☐	Avaliação e Pesquisa de Desempenho
☐	CRM

Selecionar onde estará a nova rotina e clicar em Novo Item



Preencher o Módulo que será substituído e trocar o tipo para “Função de Usuário”

Desc Português: Nome que aparecerá no menu

Programa: Mesmo nome da user function

User Function TestesTI()

Configuração do item

Desc. Português: (highlighted with a red arrow)

Desc. Espanhol:

Desc. Inglês:

Programa: (highlighted with a green arrow)

Status:
☒ Habilitado
☐ Desabilitado
☐ Inibido

Módulo: (highlighted with a red box)

Tipo: (highlighted with a red box)

Palavras-Chave Português:

Palavras-Chave Espanhol:

Palavras-Chave Inglês:

Rotinas do browse:

☒	01
☒	02
☒	03
☒	04
☒	05
☒	06
☒	07
☒	08
☒	09
☒	10

Tabelas:

Alias	Descrição
☐ A00	Território X Nível do Agrup.
☐ A01	Exceção Territoriais X Agrup.
☐ A02	Exceção Por Níveis do Agrupado
☐ A03	Sequenciador
☐ A04	Sequencia X Agrupador
☐ A05	Script X Rotina
☐ A06	Item Script Validação
☐ A08	Agrup. X Regras Para Pesquisas
☐ A09	Membros X Território
☐ A0A	Equipe X Usuarios
☐ A10	Cadastro Conjunto de Etapas
☐ A11	Checklist do Funcionário
☐ A12	Tabela Classificadora Dos doc

Busca por alias:

Criar Usuário

TOTVS | Configurador

Usuário - Cadastro de usuários

Usuário

Restrições de acesso

Dados do usuário

Usuário*
TestesTI

Nome completo*
TestesTI

Senha*
.....

Confirme a senha*
.....

Data de bloqueio(validade)
/ /

E-mail

Cargo
Superior

Tipo do bloqueio

Grupos

Superior

Adicionar o Menu Customizado no Usuário

Usuário - Cadastro de usuários

Usuário

Restrições de acesso

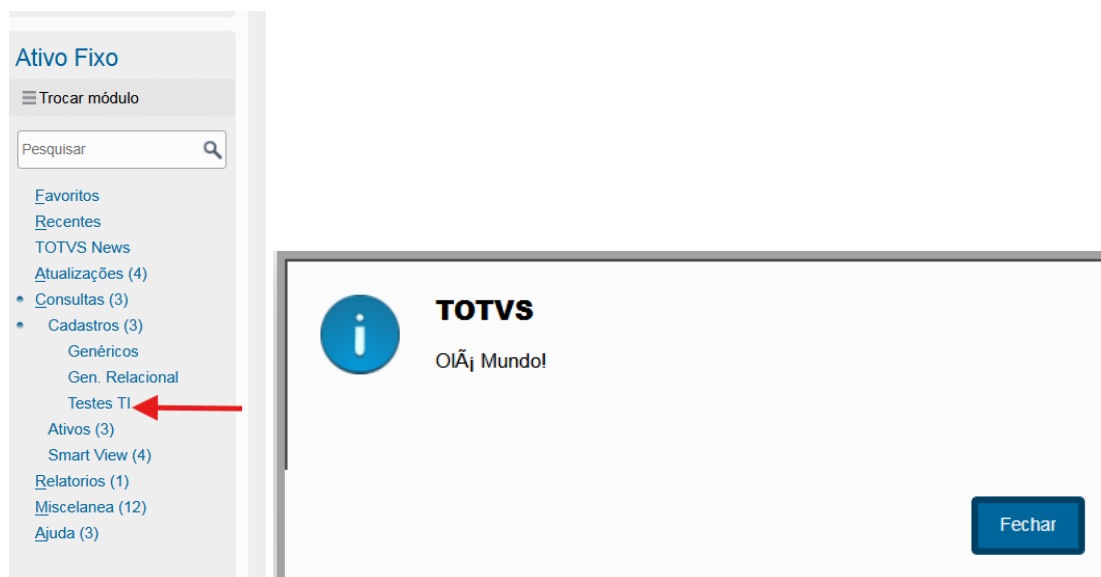
Parâmetros de restrição de acesso

Número de acessos simultâneos
2

Timeout da estação (em minutos)
0

Horário		Filiais		Ambientes
Acesso	Código	Módulo	Descrição	
☒	1	SIGAAATF	Ativo Fixo	TESTESTI
☒	2	SIGACOM	Compras	APROVACAO
☒	3	SIGACON	Contabilidade	
☒	4	SIGAEST	Estoque/Custos	SIGAEST
☒	5	SIGAFAT	Faturamento	SIGAFAT
☒	6	SIGAFIN	Financeiro	SIGAFIN
☒	7	SIGAGPE	Gestão de Pessoal	SIGAGPE
☒	8	SIGAFAS	Faturamento Serviço	
☒	9	SIGAFIS	Livros Fiscais	SIGAFIS

Acessar o Ambiente com o novo usuário e acessar a nova rotina.



27/07/2026

Tabelas

O que são SX2, SX3 e SIX no Protheus

SX2 — Cadastro de Tabelas

Define as tabelas do sistema.

SX3 — Cadastro de Campos

Define os campos das tabelas.

SIX — Índices

Define os índices utilizados nas consultas.

SX2 - Tabelas de Dados Passo 1

A tabela SX2 do Dicionário de Dados define de uma forma padronizada as tabelas disponíveis dentro do Ambiente Microsiga Protheus. É por meio desta tabela que temos o

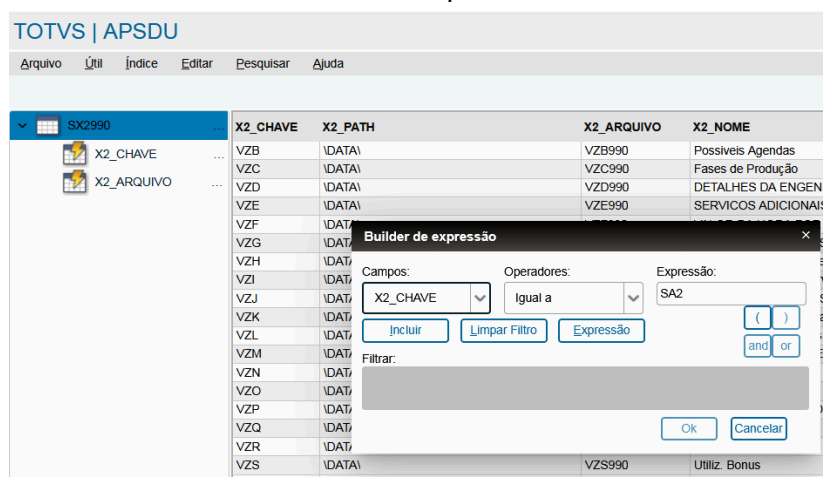
conhecimento das tabelas disponíveis, quais o alias associado a ela, caminho físico da tabela(quando CodeBase).

A tabela SX2 ainda é responsável pelas informações de modo de acesso da tabela (se compartilhado ou exclusivo).

Campos	Tipo	Descrição
X2_CHAVE	Caracter	Código da Alias da tabela, deve ser único no SX2, essa Alias será utilizada para quando formos referenciar essa tabela no código fonte;
X2_PATH	Caracter	Caminho (em relação ao rootpath) onde a tabela será gravada fisicamente, normalmente está em branco, pois só utilizada quando utilizamos banco de dados ISAM.
X2_ARQUIVO	Caracter	Nome do arquivo físico da tabela(caso esteja utilizando ISAM) ou nome da tabela se estivermos em um banco relacional(TopConnect). Segue o padrão de X2_CHAVE + Código da Unidade de negocio + 0 (Caracter 0)
X2_NOME	Caracter	Rótulo com a descrição da pergunta no idioma Português
X2_NOMESPA	Caracter	Rótulo com a descrição da pergunta no idioma Espanhol
X2_NOMEENG	Caracter	Rótulo com a descrição da pergunta no idioma Inglês
X2_ROTINA	Caracter	Rotina executada na abertura da tabela
X2_MODALO	Caracter	Conjunto de campos que determina como as informações das tabelas serão gravadas e acessadas. Valores permitidos são "C" para compartilhado e "E" para exclusivo, para maiores detalhes verifique a baixo a documentação sobre isso.
X2_MODALOUN	Caracter	
X2_MODALOEMP	Caracter	
X2_DELETE	Inteiro	*** Não usado ***

<https://tdn.totvs.com/display/public/framework/SX2+-+Tabelas+de+Dados>

No Protheus, através do APSDU, podemos acessar a SX2 e filtrar por tabelas.



<https://youtu.be/9UdKrZauvrs?si=CNvpXGjR8UHyO> Up

Adicionar uma Tabela

Podemos adicionar uma tabela, através do configurador - SIGACFG

<https://protheusadvpl.com.br/sx2-sx3-six-protheus/>

Após entrar no Configurador, entremos em **Base de Dados->Dicionário->Base de Dados**
Na próxima tela clique em **Dicionário de Dados** e no sinal de '+' conforme mostra a imagem abaixo:



Na próxima tela informamos o prefixo, no exemplo informamos o prefixo SZ1, que é o identificador da tabela no arquivo de dados SX2, automaticamente o sistema gera o nome da tabela composto pelo prefixo informado mais o código da empresa 99, assim a tabela terá seu nome como SZ1990, após isso clique para salvar as informações:

Prefixo: SZ1

Path:

Nome: SZ1990

Descrição: tabela teste

Desc. Esp.: tabela teste

Desc. Inglês: tabela teste

Ac. Filial: 1 - Compartilhado

Ac. Unidade: 1 - Compartilhado

Ac. Empresa: 1 - Compartilhado

Rotina:

Display:

Obj. Sistema:

Obj. Usuário:

TOTVS PDV:

Memo CLOB:

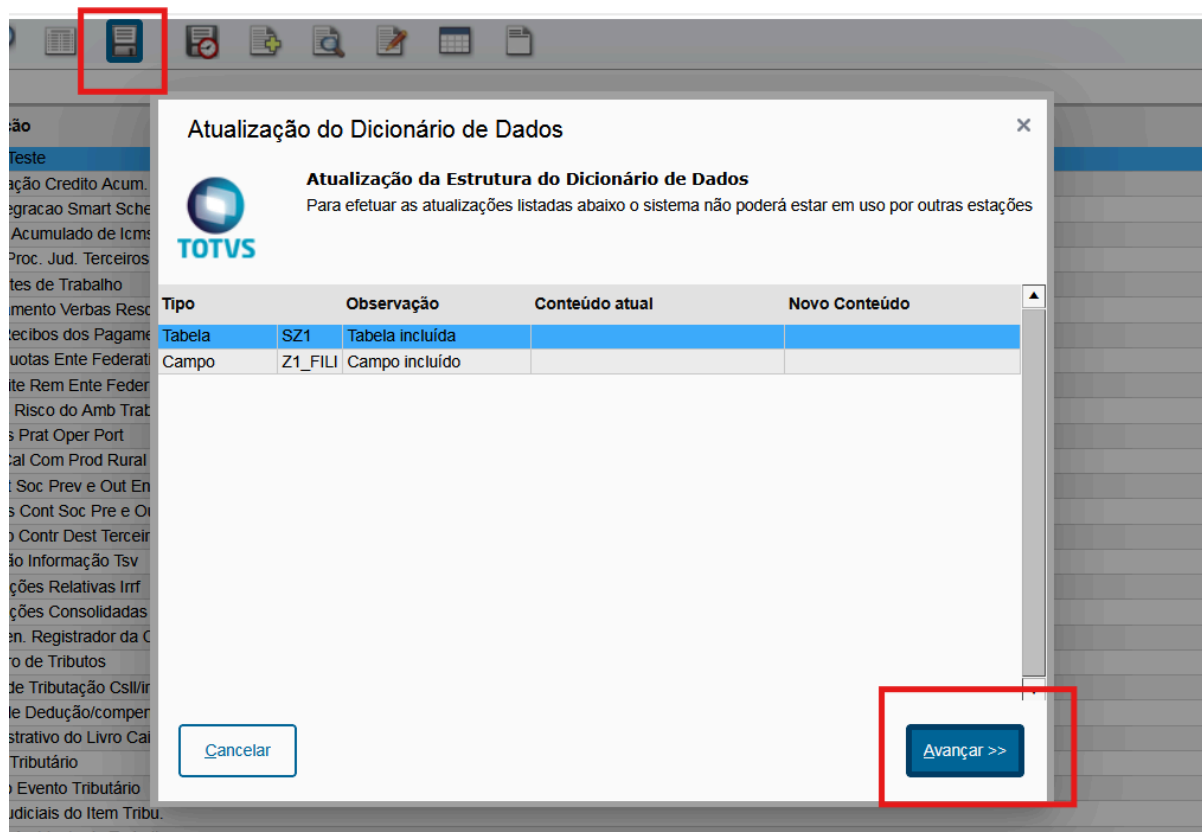
Rec.aut.Inc:

Campo STAMP:

Campo INSDT:

Chave única:

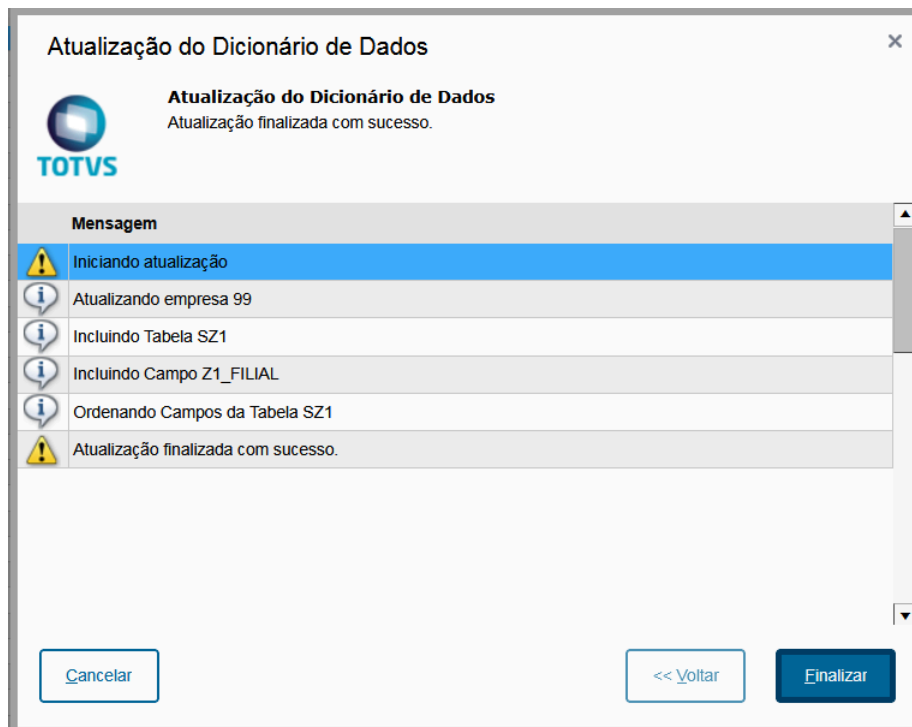
Na próxima tela clique para **salvar** as informações da tabela e o protheus criar a mesma, por padrão o protheus criar a tabela com já com o campo prefixo_FILIAL, no caso EST_FILIAL, após isso clique em avançar:



Na próxima tela o protheus informa a tabela e o campo criado, após isso clicamos em **avançar**:



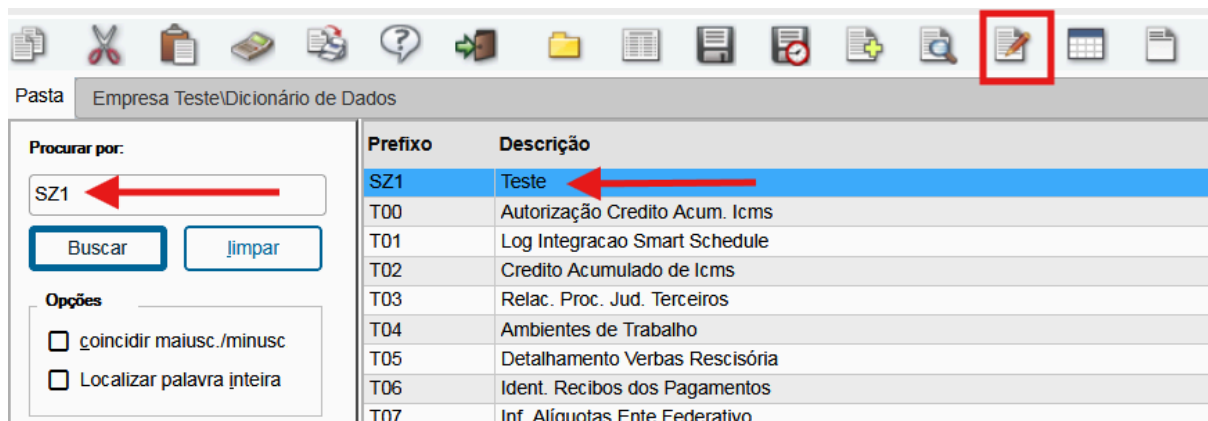
Na próxima tela o protheus mostra a atualização, a tabela e o campo criado, clique em **finalizar**:



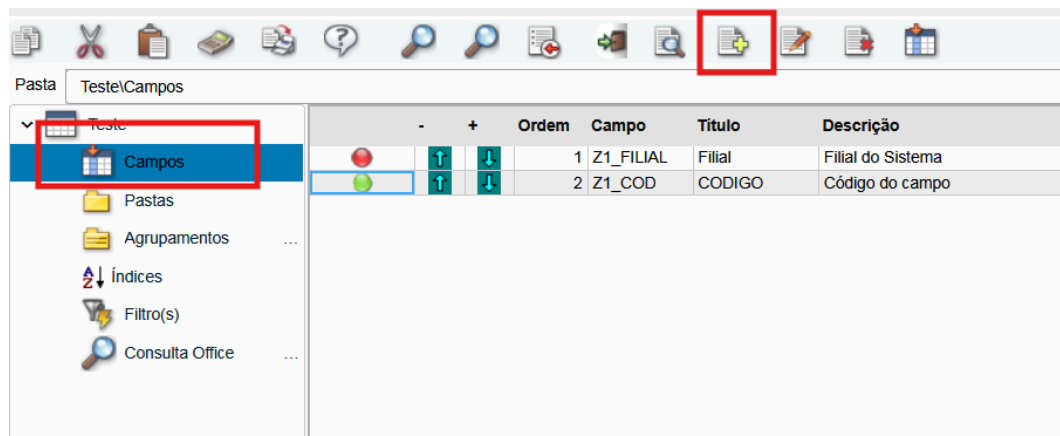
Se der erro, reiniciar o ambiente e tentar novamente.

SX3 - Criar Campos Passo 2

Na próxima etapa criaremos os campos(arquivos SX3) da tabela que criamos, para isso pesquisamos o **prefixo SZ1**, referente à tabela que criamos e clique em **editar**:



Na próxima tela clique na tabela(arquivo SX2) criada e em **campos**, após isso clique em **editar** para criar os campos:



The screenshot shows the 'Editar Campo - Z1_COD' dialog box in the 'Gerenciador de Base de Dados'. The 'Informações' tab is selected. The dialog has several fields and buttons. Red arrows point to the following fields:

- Campo: Z1_COD
- Tipo: 1 - Caracter
- Tamanho: 6
- Decimal: 0
- Contexto: 1 - Real
- Propriedade: 2 - Visualizar

Buttons at the top right include 'Outras Ações', 'Cancelar', and 'Salvar'.

Na próxima aba, em **Informações**, informamos o título do campo(obrigatório), além da descrição(opcional) e o help(opcional), que conterá informações mais detalhadas do campo.

Editar Campo - Z1_COD

Gerenciador de Base de Dados

Campo **Informações** **Opções** **Validações** **Uso** **Módulo**

Tít. Português	CODIGO
Tít. Espanhol	CODIGO
Tít. Inglês	CODIGO
Desc. Português	Código do campo
Desc. Espanhol	Código do campo
Desc. Inglês	Código do campo
Help	Código sequencial do campo

Na próxima aba, em Opções, na campo Inic. Padrão(inicializador padrão) passamos a função **GETSX8NUM(<alias>, <campo>)**, utilizada para obtermos o próximo número disponível para o alias especificado no parâmetro, assim quando informamos GETSX8NUM("SZ1", "SZ1_CODIGO") estamos querendo obter o próximo número referente ao valor contido no campo SZ1_CODIGO, assim cada vez que formos fazer um cadastro nessa tabela o protheus verifica qual o valor contido no campo SZ1_CODIGO, supondo ser o valor '000007', ele obterá o próximo '000008', obtendo o número sequencial a cada novo cadastro:

Editar Campo - Z1_COD

Gerenciador de Base de Dados

Campo **Informações** **Opções** **Validações** **Uso** **Módulo**

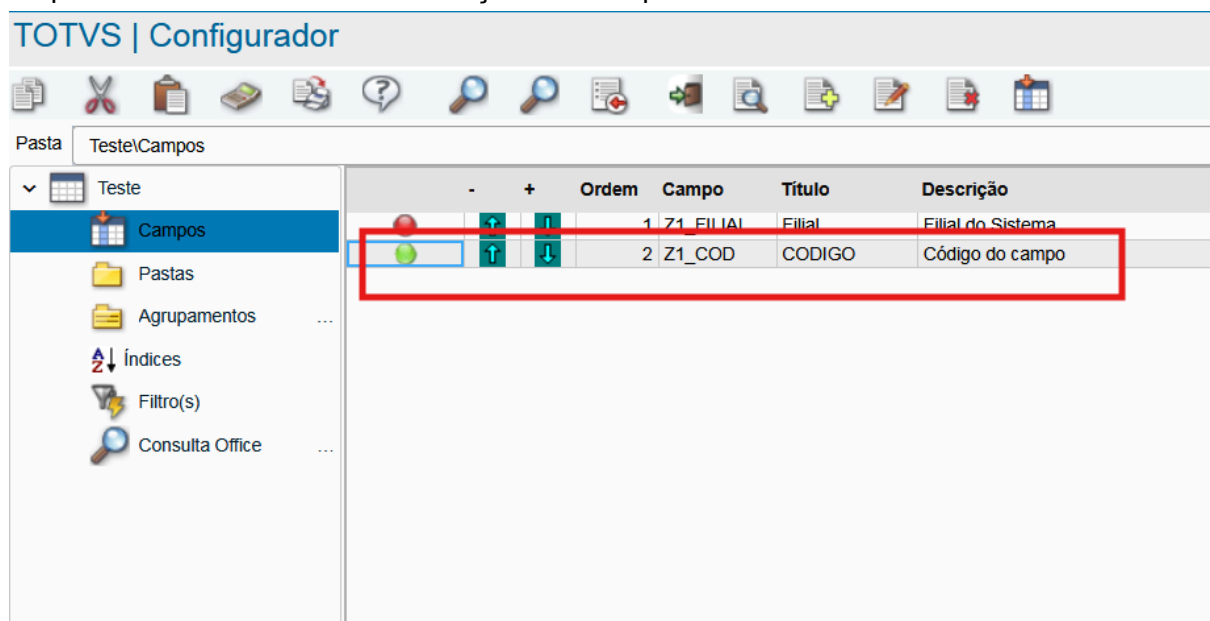
Lista Opções	
Lista Espanhol	
Lista Inglês	
Inic. Padrão	GETSX8NUM(<ALIAS>, <CAMPO>) ←
Inic. Browse	
Modo Edição	
Corretor	☐
P Search	☐
Cons. Padrão	

Na aba Uso deixamos marcado os box usado e browse, indicando que o protheus irá usar esse campo e o no browse indicamos ao protheus que mostre esse campo na abertura da tela:

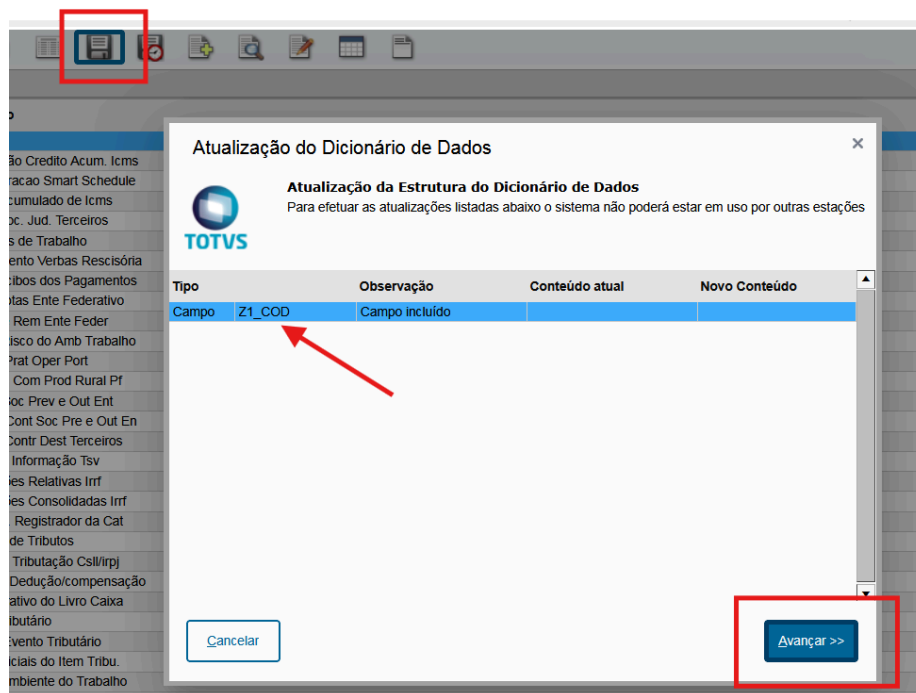


Após isso clique em **Salvar**

Na próxima tela confirme as informações do campo



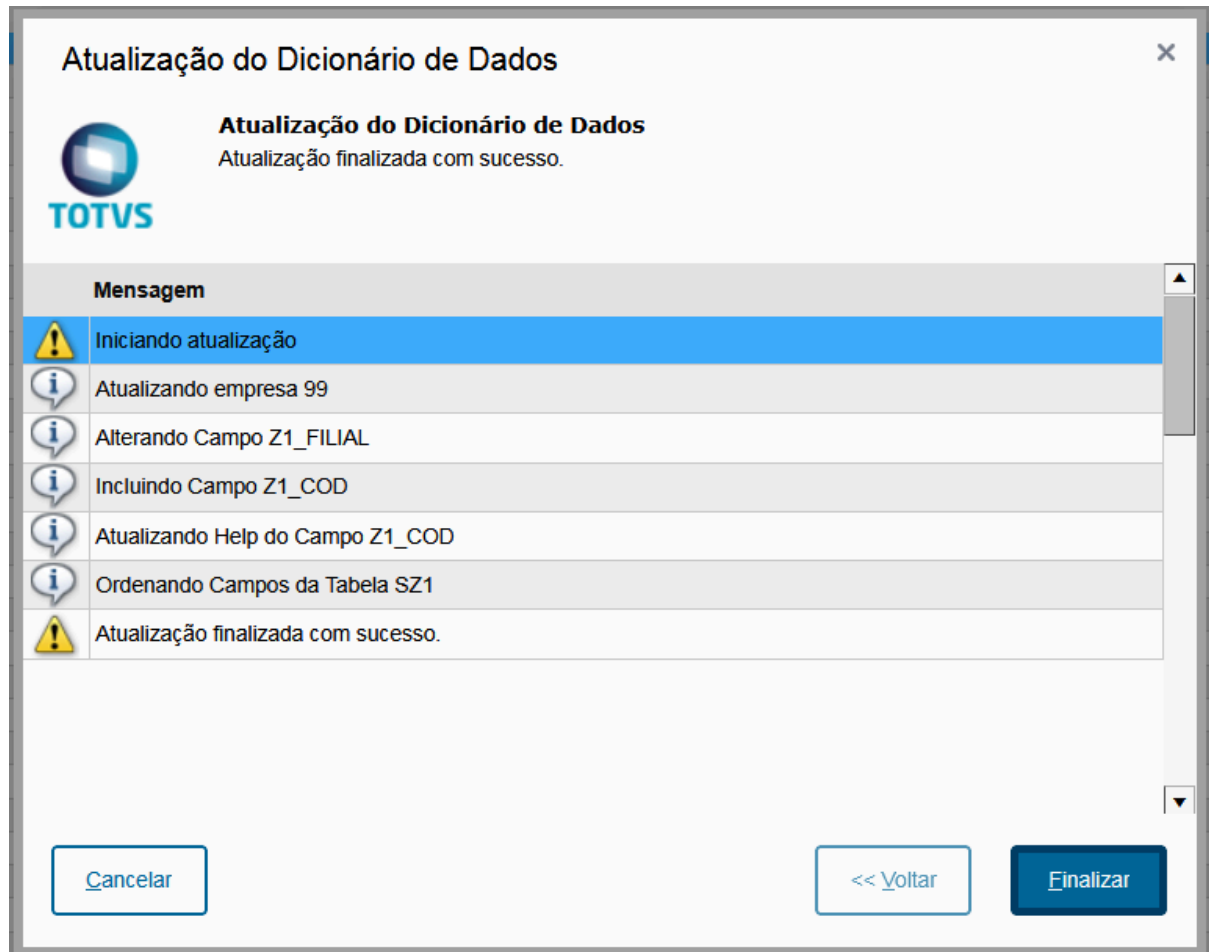
Na próxima tela clique no símbolo de disquete para salvar as informações e clique em avançar, conforme você ver na imagem abaixo, aparece mais dois campos além do que criamos, eu criei ele apenas para preencher a tela com mais campos quando formos visualizarmos a mesma



Na próxima tela visualizamos os campos a serem incluídos e clicamos em **avançar**:



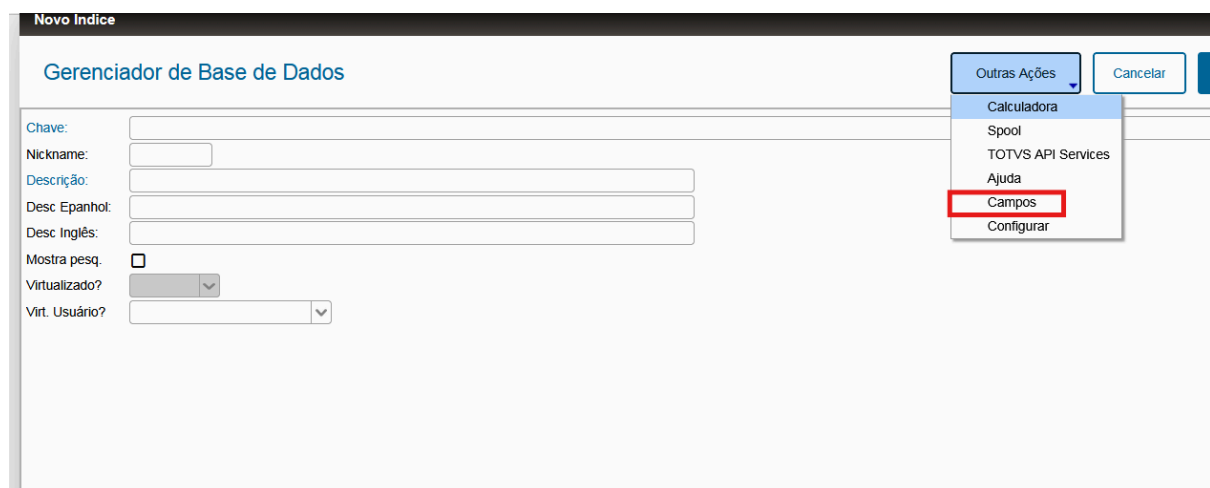
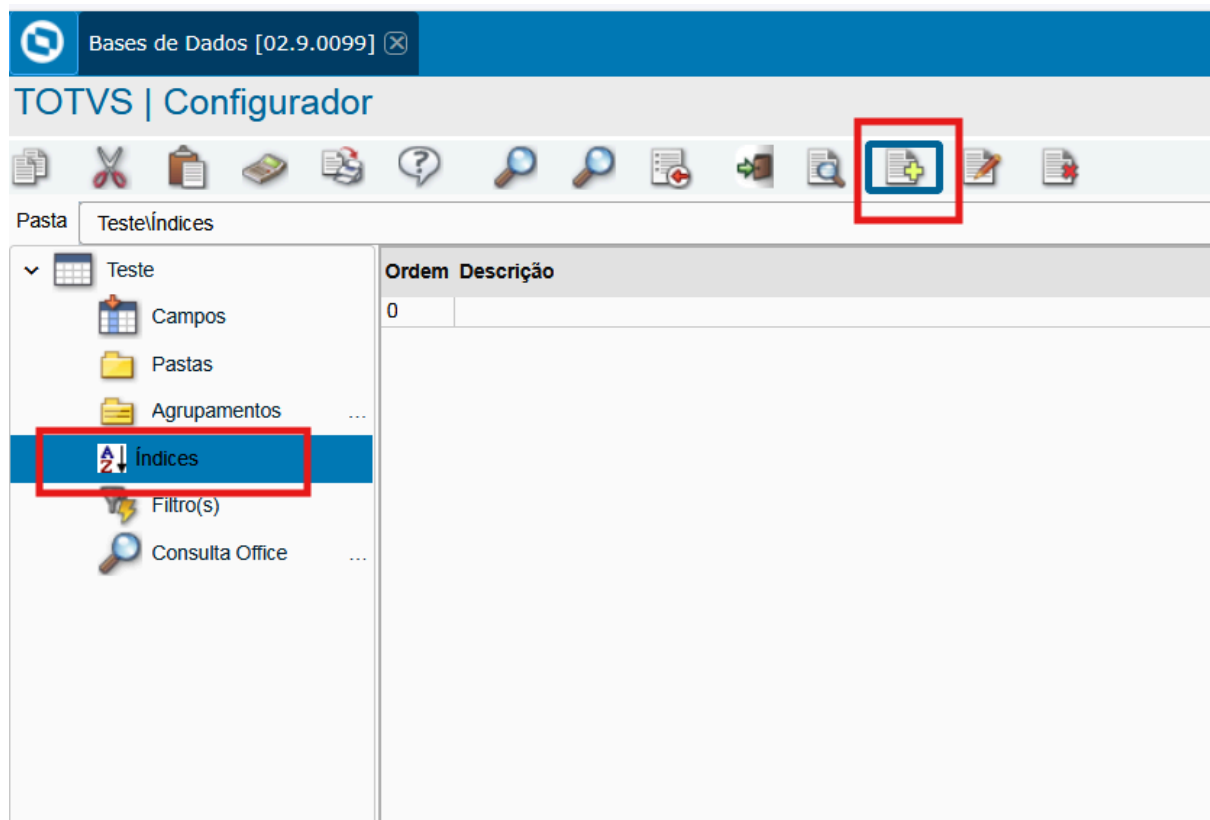
Na próxima tela visualizamos os campos criados pelo protheus, clique em **finalizar**:



Tabelas criadas:

	-	+	Ordem	Campo	Título	Descrição
			1	Z1_FILIAL	Filial	Filial do Sistema
			2	Z1_COD	CODIGO	Código do campo
			3	Z1_DESC	Descricao	Descricao do campo
			4	Z1_QTDE	Quantidade	Quantidade
			5	Z1_DATA	Data	Data
			6	Z1_VALOR	Valor	Valor

SIX Criar Índices



Consultar Campos

Z1_FILIAL

Z1_COD

Z1_DESC

Z1_QTDE

Z1_DATA

Z1_VALOR

OK

Cancelar

Novo Índice

Gerenciador de Base de Dados

Chave:

Z1_FILIAL+Z1_COD

Nickname:

Descrição:

CODIGO

Desc Epanhol:

CODIGO

Desc Inglês:

CODIGO

Mostra pesq.

☐

Virtualizado?

▼

Virt. Usuário?

▼

Para usar uma variável, primeiro precisamos declará-la. A forma mais comum e recomendada de fazer isso em ADVPL é usando a palavra-chave **Local**.

Declarar uma variável como **Local** significa que ela só existirá dentro do escopo da função onde foi criada. Isso evita que ela interfira ou seja modificada por outras partes do sistema, o que é uma boa prática de programação.

A sintaxe é simples: **Local <nome_da_variavel> := <valor_inicial>**

Variáveis e Tipos de Dados no ADVPL

Declaração de Variáveis

`</> Local cNome := "João"`→ (String / Caractere)

`</> Local nIdade := 25`→ (Numeric / Número)

`</> Local lAtivo := .T.`→ (Logical / Lógico)

`</> Local dData := Date()`→ (Date / Data)

`</> Local aItens := {}`→ (Array / Matriz/Lista)

Prefixos e Tipos

c = Character (texto)
Armazena seqüências de caracteres alfanuméricos.

123 n = Numeric (número)
Armazena números inteiros ou decimais.

I = Logical (verdadeiro/falso)
Armazena valores booleanos .T. ou .F.

d = Date (data)
Armazena datas no formato DD/MM/AAAA.

[...] a = Array (lista)
Armazena coleções ordenadas de elementos de qualquer tipo.

NOTA IMPORTANTE: ADVPL é fracamente tipado - o tipo é definido pelo valor atribuído à variável!

<https://dicionario.dev.br/guias/variaveis-tipos-advpl>

Os 5 Tipos de Dados Fundamentais

O ADVPL possui vários tipos de dados, mas cinco deles são a base para quase tudo que você fará no dia a dia. É crucial entender o que cada um representa e como usar seus prefixos de nomenclatura.

Prefixo	Tipo	Descrição	Exemplo de Valor
c	Caractere	Armazena texto, como nomes, descrições, códigos. O valor fica entre aspas.	"Olá, Mundo!"
n	Numérico	Armazena números, sejam eles inteiros ou com casas decimais.	10, 25.99, -50
l	Lógico	Armazena valores de Verdadeiro ou Falso.	.T. (True), .F. (False)
d	Data	Armazena datas no formato padrão do Protheus (geralmente DD/MM/AAAA).	dDataBase

a	Array	É uma coleção ou lista que pode armazenar múltiplos valores, inclusive de tipos diferentes.	{"Maçã", "Banana", "Uva"}
---	-------	---	---------------------------

Operadores Matemáticos

Operador	Descrição
+	Adição
-	Subtração
/	Divisão
** ou ^	Exponenciação
%	Resto da divisão(Módulo)

Operadores de String

Operador	Descrição
+	Concatenação de strings (união)
-	Concatenação de strings com eliminação dos brancos finais das strings intermediárias
\$	Comparação de Substrings (contido em)

Operadores Relacionais

Operador	Descrição
<	Comparação Menor
>	Comparação Maior
=	Comparação Igual

==	Comparação Exatamente Igual
<=	Comparação Menor ou Igual
>=	Comparação Maior ou Igual
<> ou !=	Comparação Diferente (*) (*) O operador '#' foi mantido por compatibilidade no AdvPL para indicar comparação por diferença, mas seu uso é obsoleto.
:=	

Operadores Lógicos

Operador	Descrição
.And.	E
.Or.	OU
.Not. ou !	NÃO

Bibliotecas Utilizadas - Includes

No início do Fonte, são declarados os includes que são as bibliotecas que serão utilizadas.

Principais Bibliotecas:

TOTVS.ch – Biblioteca padrão para as customizações

Protheus.ch – Antiga biblioteca padrão do Protheus, deve-se utilizar por padrão a TOTVS.ch

TopConn.ch – Biblioteca com funções e recursos para usar com conexões via Top Connect (como a famosa TCQuery)

<https://terminaldeinformacao.com/2020/06/28/quais-sao-as-principais-bibliotecas-em-advpl/>

User Function (ou U_)

Indica uma função customizada, criada por desenvolvedores ou clientes. O Protheus reconhece essa rotina como uma extensão do sistema padrão.

ximpSZ1: É o nome da função (um padrão comum é usar **x** ou a sigla da empresa antes do nome da rotina para indicar que é customizada).

(): Lista de parâmetros que a função aceita. Nesse caso, está vazia (não recebe parâmetros).

```

1  #include "totvs.ch"
2  #include "protheus.ch"
3  #include "TOPCONN.CH"
4
5  > user function ximpSZ1() // U_uReadExc() ...
20 > // Local cCodUse := RetCodUsr() ...
104 return
105

```

Em vermelho: Includes

Em verde: User function

Declaração de Variáveis: Local ...

Em AdvPL, variáveis locais são declaradas usando **Local** e duram apenas durante a execução desse bloco/função. É uma boa prática usar a notação **Hungarian (Húngara)**, em que o prefixo indica o tipo do dado:

```

2  #include "protheus.ch"
3  #include "TOPCONN.CH"
4
5  0 reference
6  user function ximpSZ1() // U_uReadExc()
7
8      //Variaveis para ler o CSV
9      local cDiret
10     local aCampos := {}
11     local aDados  := {}
12     local aLinha  := {}
13     Local AxSZ1IMP := {}
14     //Variaveis para mapear os campos
15     //Local lincount := 1
16     Local lPrimLin := .T.
17     Local NJX      := 1
18     Local nProc     := 0
19     Local nAtual    := 0
20     local qtdaux    := 0

```

Variáveis para ler o csv

	c (Character/String)	cDiret	Armazena o caminho do arquivo selecionado no computador (ex: "C:\pasta\arquivo.csv").
	Array	aCampos := {}	Array para guardar o cabeçalho separado do CSV.
	Array	aDados := {}	Array temporário para guardar os dados de uma linha específica.
	Array	aLinha := {}	Array temporário para separar os dados contidos em uma linha lida
	Array principal (matriz)	AXSZ1IMP := {}	É onde todas as linhas válidas do arquivo serão armazenadas na memória antes de irem para o banco de dados.
	l (Logical/Booleano - .T. ou .F.)	lPrimLin := .T.	Flag (sinalizador) indicando que a rotina está processando a primeira linha do arquivo (geralmente usada para validações de cabeçalho).
	n (Numeric/Número)	NJX := 1	Variável de controle para laços de repetição (For...Next).
	n (Numeric/Número)	nProc := 0	Guardará a quantidade total de processos/registros.
	n (Numeric/Número)	nAtual := 0	Contador para controlar qual registro está sendo gravado no momento.
	n (Numeric/Número)	qtdaux := 0	Variável auxiliar para armazenar o tamanho total do array (Len(AXSZ1IMP)).

Funções nesse arquivo

Seleção de Arquivo: cGetFile(...)

A função cGetFile abre uma janela gráfica no sistema para que o usuário navegue e escolha um arquivo local ou da rede.

```

29 //
30 //Abrir uma tela para escolher o arquivo.
31 cDiret := cGetFile('Arquivo CSV|*.csv| Arquivo TXT|*.txt| Arquivos XML|*.xml',; //Máscara e extensão dos arqv que o usuário poderá visualizar.
32 'Selecao de Arquivos',; //cTitulo da janela pop-up.
33 0,; //nMascpadrao Filtro inicial padrão selecionado na lista.
34 'C:\csv\',; //cDirinicial Pasta inicial em que o navegador de arquivos abrirá.
35 .F.,; //lSalvar Modo da janela: .F. botão de Abrir | .T. botão de Salvar).
36 GETF_LOCALHARD + GETF_NETWORKDRIVE,; //nOpcoes Permite buscar arqv no disco local e em pastas mapeadas de rede.
37 .T.) //Exibe o diretório do servidor de aplicação ou local.
38
39 //O caminho final do arqv escolhido pelo usuário é retornado e gravado dentro da variável cDiret
40
41 FT_FUSE(cDiret) //Disponibiliza as funções FT_F para o usuário

```

Função	Exemplo de uso	Descrição
FT_FUSE	FT_FUSE(cDiret)	Abre/carrega na memória do Protheus o arquivo indicado pelo caminho salvo em cDiret para disponibilizar às funções FT_F (File Text).
FT_FLASTREC	ProcRegua(FT_FLASTREC())	Lê e retorna o número total de linhas do arquivo texto aberto pela função FT_FUse()
FT_FGOTOP	FT_FGOTOP()	Posiciona no início (primeiro caractere da primeira linha) do arquivo texto aberto pela função FT_FUse()
FT_FEOF	While !FT_FEOF()	Move o ponteiro, que indica a leitura do arquivo texto, para a posição absoluta especificada no parâmetro
FT_READLN	aLinha := Separa(FT_FREADLN(),",",.T.)	Lê e retorna uma linha de texto do arquivo aberto pela função FT_FUse(). As linhas do texto.
FT_FSKIP	FT_FSKIP()	Move o ponteiro, do arquivo texto aberto pela função FT_FUse(), para uma nova posição.
ProcRegua	ProcRegua(FT_FLASTREC())	Antes de entrar no laço (While), o código conta quantas linhas o arquivo possui no total.
IncProc	IncProc('Lendo Arquivo texto...')	A cada iteração do While, o IncProc() avança um "passinho" nessa régua visual e exibe a mensagem 'Lendo Arquivo texto...' para o usuário saber que o sistema não travou.
alltrim	IF alltrim(aLinha[1]) == "SZ1"	Remove todos os espaços em branco antes e depois do texto (evitando que um espaço como "SZ1 " falhe na comparação)
Len()	Len(AXSZ1IMP)	A função Len() conta quantos elementos/linhas existem dentro do array AXSZ1IMP

Função para Leitura do Arquivo

```
38 //O caminho final do arqv escolhido pelo usuário é retornado e gravado dentro da variável cDiret
39
40 //Leitura do arquivo
41 FT_FUSE(cDiret) //Disponibiliza as funcoes FT_F para o arquivo. Abre o arquivo localizado no caminho guardado na variável cDiret e avisa o Protheus:
42 //Prepara a biblioteca FT_F pra manipular esse arquivo"
43 ProcRegua(FT_FLASTREC()) //FT_FLASTREC() conta o número total de linhas do arquivo. | ProcRegua(...) pega esse número total e configura a régua do Protheus.
44 //Exemplo: se o arquivo tem 100 linhas, a régua sabe que 100 linhas equivalem a 100% da barra de progresso.
45 FT_FGOTOP() //Seta a primeira linha do arquivo
46
47 While !FT_FEOF() //While enquanto nao for o final do arquivo (End Of File)
48 IncProc("Lendo Arquivo texto...") //A cada volta do While, anda 1 "passinho" na barra de progresso da tela e mostra essa mensagem.
49 // cLinha := FT_FREADLN() //Le a linha atual e coloca ela na cLinha
50 aLinha := Separa(FT_FREADLN(),";",".T.) //FT_FREADLN(): lê a linha onde o ponteiro ta e devolve o texto inteiro dessa linha como uma única string |
51 //Separa(): Pega esse texto inteiro e o "fatia" onde tiver o delimitador ponto e vírgula (;), transformando-o num Array (vetor) de pedaços. |
52
53 //validação da primeira linha do arquivo
54 If lPrmlin
55 IF alltrim(aLinha[1]) == "SZ1" //Testa se a flag lPrmlin está verdadeira (ela começa .T.). Isso garante que este bloco só vai rodar na primeira linha do arquivo.
56 lPrmlin := .F. //pega o primeiro pedaco que a função Separa() fatiou | alltrim(...) tira qualquer espaço em branco das pontas (caso esteja "SZ1 " ou " SZ1").
57 lcabok := .T. //Desliga" a flag de primeira linha. Nas próximas voltas do While, o sistema pula esse bloco If lPrmlin.
58 FT_FSKIP() //Liga" a flag de cabecalho ok. Isso vai permitir que o sistema entre no próximo bloco If lcabok.
59 //Pula para a próxima linha do arquivo. Isso é necessário porque a primeira linha do arquivo é só o cabecalho, não tem dados.
60 Else
61 Alert("Corrija a importacao para tabela SZ1")
62 Return
63 Endif
64 Endif
```

```
66 If lcabok
67 aCampos := Separa(FT_FREADLN(),";",".T.) //Se a flag do cabecalho está autorizada (o bloco anterior confirmou que o arquivo começa com sz1)
68 //Caso seja a primeira, seta no vetor aCampos[], separando os campos por ;
69 //FT_FREADLN(): Lê a segunda linha do arquivo (que contém "COD;DESC;QTD;DATA;VALOR").
70 //Separa(..., ";", ".T.): Fatia essa linha e gera o vetor aCampos
71 //Conferindo se o nome de cada coluna da segunda linha está escrito exatamente na ordem esperada
72 IF ((aCampos[1] == "COD") .AND. ; //carac
73 (aCampos[2] == "DESC") .AND. ; //carac
74 (aCampos[3] == "QIDE") .AND. ; //num
75 (aCampos[4] == "DATA") .AND. ; //data
76 (aCampos[5] == "VALOR")) //num
77 lcabok := .F. //Desliga a flag de verificação do cabecalho. Isso impede que este bloco If lcabok rode novamente quando o loop ler a linha 3 (dados dos produtos)
78 FT_FSKIP() //Pula para a próxima linha do arquivo. Isso é necessário porque a segunda linha do arquivo é só o cabecalho, não tem dados.
79 else
80 Alert("Cabecalho da tabela não foi encontrado, campos Descrição | Quantidade | Data | Valor")
81 return
82 endif
83 endif
84
85 adados := Separa(FT_FREADLN(),";",".T.) //Lê a linha corrente de dados
86 //Separa(..., ";", ".T.) fatia a string nos pontos e vírgulas e joga na variável adados
87 aadd(AXSZ1IMP,adados[1],adados[2],adados[3],adados[4],adados[5]) //adiciona um novo elemento/linha ao final de um Array
88 //adados[1], ...): Cria uma sub-lista com os 5 valores da linha atual e insere dentro do vetor AXSZ1IMP
89 //O array AXSZ1IMP agora guarda todos os produtos carregados do arquivo antes de fazer a gravação oficial no BD
90 //Move o ponteiro do arquivo para a próxima linha (linha de baixo) do CSV
91 FT_FSKIP()
92 EndDo
```

```
SZ1->(dbSetOrder(8))
if SZ1-> (DBSEEK(XFILIAL("SZ1") + Padr(AXSZ1IMP[njx][1], TamSX3("SZ1_CODPRO")
[1])+Padr(AXSZ1IMP[njx][2],TamSX3("SZ1_CODMON")([1])))
```

O que o **IF** faz: Tenta encontrar se o registro já existe no banco.

- Se encontrar (Retorna **.T.**): Entra no bloco **If** para alterar o registro.
- Se NÃO encontrar (Retorna **.F.**): O código atual ignora e não grava nada (pois não há um **Else** para inclusão).

Funções de Conversão

Nome	Descrição
------	-----------

CTOD()	Converte Caractere para Date no formato DD/MM/AAAA
DTOC()	Converte Data para Caractere DD/MM/AA
DTOS()	Converte Data para String formato AAAAMMDD
STOD()	Converte String para Data formato AAAAMMDD
CVALTOCHAR()	Converte Caractere, Data,Lógico ou Num para String sem adição de espaços
STR()	Converte Numérico para String com espaços à esquerda e/ou símbolo decimal(.) nas casas
STRZERO()	Converte Numérico em String com zeros à esquerda para completar o tamanho da variável sintaxe (nValor, nTamanho, nDecimal)
VAL()	Converte sequência de caracteres que contém dígitos em valor numérico: 12-> 12.00

Funções de String

Nome	Descrição
ALLTRIM()	Remove espaços à direita e a esquerda
RTRIM()	Remove espaços à direita
LTRIM()	Remove espaços à esquerda
ASC()	Converte valor de caractere com a tabela ASCII
AT()	Retorna a primeira posição/ocorrência de um caractere ou string, dentro de outra string
RAT()	Retorna a última posição de um caractere ou string, dentro de outra string específica
CHR()	Função que retorna um caractere conforme código ASCII
LEN()	Retorna o tamanho de uma variável
LOWER()	Retorna uma string com todos os caracteres minúsculos
UPPER()	Retorna uma string com todos os caracteres maiúsculos
SUBSTR()	Pega parte de uma string, recebe a posição

	inicial, e a quantidade de caracteres que irá buscar.
STUFF()	Substitui um caractere em uma string já existente, especificando a posição inicial e o n° de caracteres a serem substituídos
PADR()	Adiciona caracteres de preenchimento à direita do conteúdo da variável, indicando o tamanho da String e qual caractere será o preenchimento
PADC()	Adiciona caracteres de preenchimento à direita e esquerda para centralizar o conteúdo da variável
PADL()	Adiciona caracteres de preenchimento à esquerda do conteúdo da variável
REPLICATE()	Gera uma String repetida a partir de outra
STRTRAN()	Pesquisa e substitui um conjunto de caracteres de uma string

Funções de impressão

Função	Descrição
MsgAlert(cMsg, cTitulo)	Exibe um triângulo amarelo de atenção
MsgInfo(cMsg, cTitulo)	Exibe um ícone azul de informação
MsgStop(cMsg, cTitulo)	Exibe um círculo vermelho com um X de erro ou parada
Alert(cMsg)	Exibe um alerta padrão simples onde o título é fixado como 'TOTVS'
FWAlertInfo(cMsg, cTitulo)	Mensagem informativa com visual moderno e ícone azul
FWAlertSuccess(cMsg, cTitulo)	Mensagem de sucesso com ícone verde
FWAlertError(cMsg, cTitulo)	Mensagem de erro com ícone vermelho
MsgYesNo(cMsg, cTitulo)	Cria uma caixa de pergunta com botões Sim e Não . Retorna .T. ou .F.
Aviso(cTitulo, cMsg, aBotoes, nSize)	Permite criar uma janela de aviso mais customizada com múltiplos botões definidos por um array

Constantes declaradas - Defines

Similar à variáveis porém seus valores não são alterados, por exemplo posições em array

https://youtu.be/jWQKdgait5o?si=PFNywT_u0V5QpNor

Operadores Comuns

<https://tdn.totvs.com/display/tec/Operadores+Comuns>

Funções de Conversão no ADVPL: CToD, DToS, SToD, cValToChar, Str, StrZero e Val

<https://youtu.be/41ddF04M-l4?si=awZ6d2cDA9utFRdK>

Como Usar SubStr, StrTran, PadR, PadL, Stuff e Outras Funções de String

<https://youtu.be/QLpXag8kacg?si=q40g9s1RLPho7OrH>